

1 Part 1: Timing Analysis

1.1 1.5 Histograms

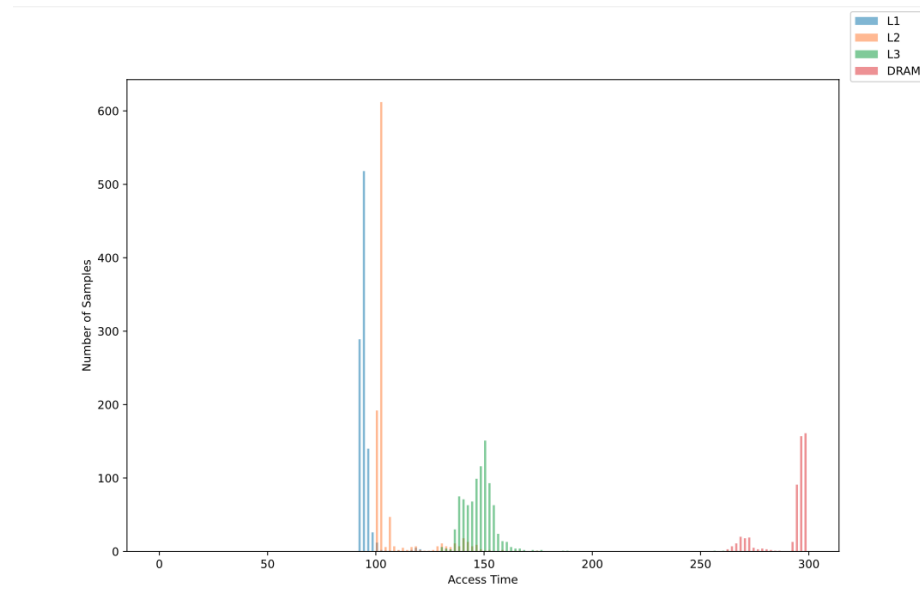


Figure 1: Reference Histogram

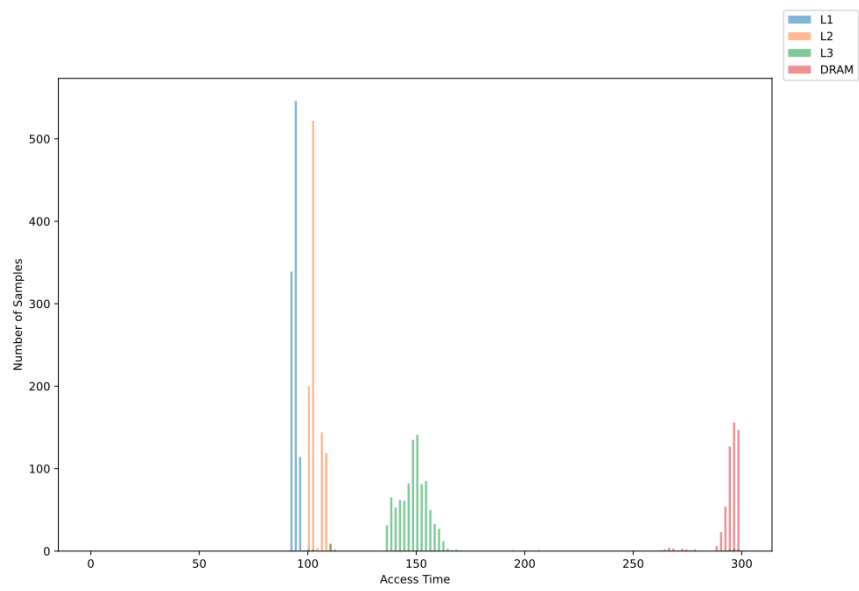


Figure 2: Main Histogram

2 Part 2: DeadDrop

2.1 Discussion Question 4

Encode The sender reads one bit ('1' or '0') and uses a 2 MB huge page as shared memory. If the bit is '1', it keeps accessing selected cache sets very fast during one time window, which creates strong cache activity. If the bit is '0', it stays mostly quiet during that window. Typing 'q' stops the sender.

Receive The receiver also maps a 2 MB huge page and repeatedly checks the same cache sets during each bit window. It measures access time, where slower time means more cache misses. For each small sample, it decides whether misses look high or low, and this gives a temporary vote for '1' or '0'.

Decode At the end of each bit window, the receiver uses majority voting from all temporary votes to decode the final bit ('1' or '0'). To reduce noise, it only prints a value after seeing the same decoded bit three times in a row. The timing itself is measured with 'measure_one_block_access_time' from 'util.c'.

2.2 Discussion Question 5

Given Data Used:

- L2 cache size = 1 MB
- L2 number of sets = 1024
- Cache line size = 64 B

Page Size	4KB	2MB
Page Offset Bits	12	21
Page Number Bits	52	43
L2 Set Index Bits	10	10
L2 Set Index Fully Under Control?	No	Yes

Table 1: Address calculation for different page sizes

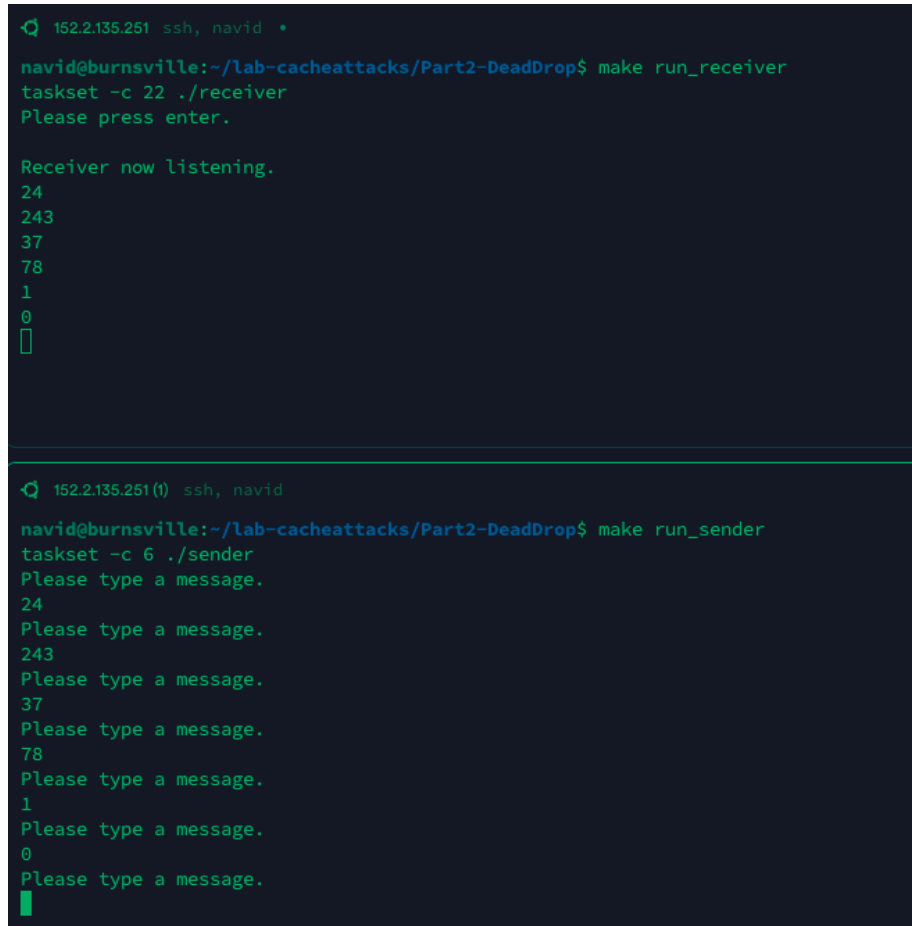
2.3 Discussion Question 6

Encode The sender reads one integer from '0' to '255', so it is an 8-bit value. It uses one group of L2 cache sets for each bit position. If a bit is '1', the sender creates cache activity in that bit's sets; if a bit is '0', it does not create activity for that bit. It repeats this many times for the same value ('SYMBOL_REPEAT') so the receiver can measure it clearly. It also sends a small marker signal on a separate marker set group so the receiver can tell the difference between real data and idle time.

Receive The receiver also allocates a 2 MB huge page and keeps checking the same set groups used by the sender. First, it checks the marker sets. If marker activity is not seen, the receiver treats that window as idle and skips decoding. If marker activity is seen, it probes each of the 8 bit set groups and counts cache misses. For each bit group, if misses are above a threshold, that bit is decoded as '1'; otherwise it is decoded as '0'. This builds one 8-bit integer value.

Decode After the receiver builds the integer, it applies a small confidence filter to reduce noise. It requires the same decoded value to appear in several consecutive windows before printing it. After printing, it briefly locks out repeated prints of the same symbol and waits for a change. This keeps output stable and prevents continuous duplicate lines when the sender is between messages.

2.4 Solution: DeadDrop



```
152.2.135.251 ssh, navid *
navid@burnsville:~/lab-cacheattacks/Part2-DeadDrop$ make run_receiver
taskset -c 22 ./receiver
Please press enter.

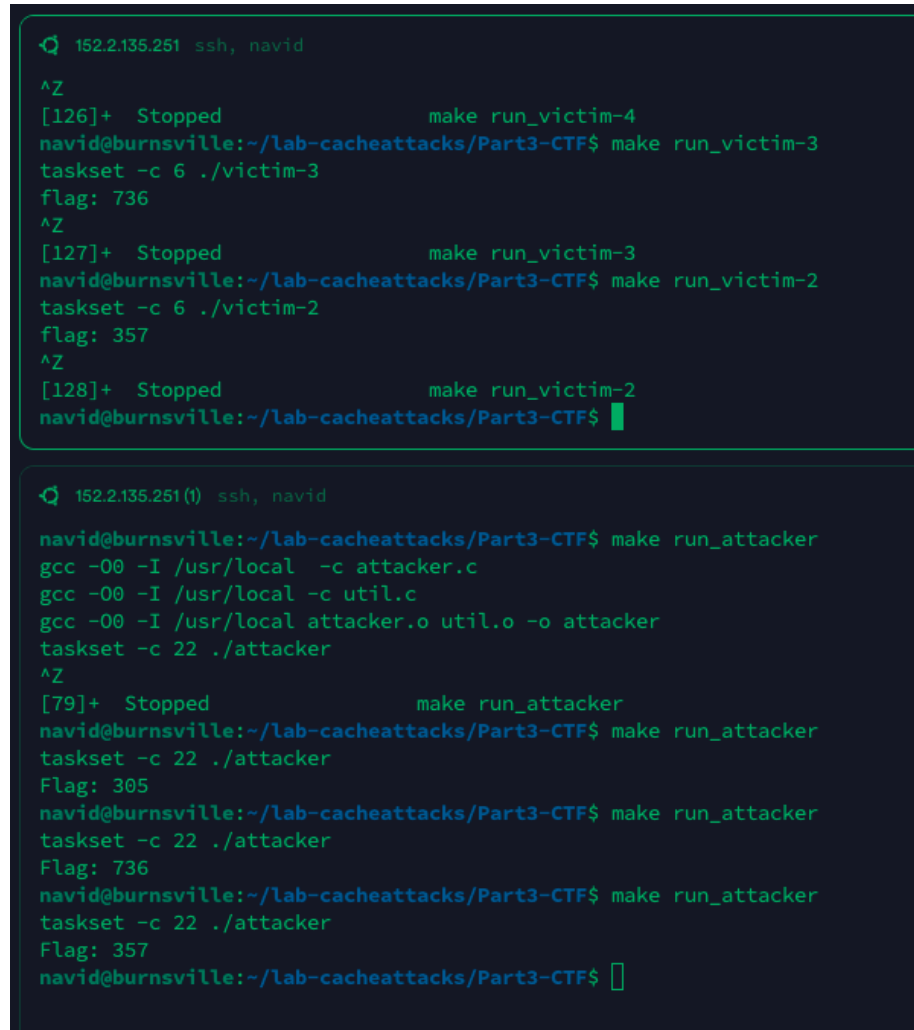
Receiver now listening.
24
243
37
78
1
0
█

152.2.135.251() ssh, navid
navid@burnsville:~/lab-cacheattacks/Part2-DeadDrop$ make run_sender
taskset -c 6 ./sender
Please type a message.
24
Please type a message.
243
Please type a message.
37
Please type a message.
78
Please type a message.
1
Please type a message.
0
Please type a message.
█
```

Figure 3: DeadDrop Solution

3 Part 3: CTF

3.1 Solution

A terminal window with a dark background and light green text. It shows a series of commands and outputs for a CTF challenge. The user is logged in as 'navid' on a machine named 'burnsville'. The directory is '~/lab-cacheattacks/Part3-CTF'. The user runs 'make run_victim-4', 'make run_victim-3', 'make run_victim-2', and 'make run_attacker' multiple times. Each time, the output shows the taskset command, the flag value, and a prompt to press ^Z to stop. The flags are 736, 357, 305, 736, and 357 respectively.

```
152.2.135.251 ssh, navid
^Z
[126]+ Stopped make run_victim-4
navid@burnsville:~/lab-cacheattacks/Part3-CTF$ make run_victim-3
taskset -c 6 ./victim-3
flag: 736
^Z
[127]+ Stopped make run_victim-3
navid@burnsville:~/lab-cacheattacks/Part3-CTF$ make run_victim-2
taskset -c 6 ./victim-2
flag: 357
^Z
[128]+ Stopped make run_victim-2
navid@burnsville:~/lab-cacheattacks/Part3-CTF$

152.2.135.251 (1) ssh, navid
navid@burnsville:~/lab-cacheattacks/Part3-CTF$ make run_attacker
gcc -O0 -I /usr/local -c attacker.c
gcc -O0 -I /usr/local -c util.c
gcc -O0 -I /usr/local attacker.o util.o -o attacker
taskset -c 22 ./attacker
^Z
[79]+ Stopped make run_attacker
navid@burnsville:~/lab-cacheattacks/Part3-CTF$ make run_attacker
taskset -c 22 ./attacker
Flag: 305
navid@burnsville:~/lab-cacheattacks/Part3-CTF$ make run_attacker
taskset -c 22 ./attacker
Flag: 736
navid@burnsville:~/lab-cacheattacks/Part3-CTF$ make run_attacker
taskset -c 22 ./attacker
Flag: 357
navid@burnsville:~/lab-cacheattacks/Part3-CTF$
```

Figure 4: CTF Solution